

Computational Identity

A Formal Theory of Occurrent and Modal Realization

Mathematical appendix to the conceptual guide

August 2026

Purpose

The formalism is designed to make four distinctions explicit. First, a function is not an algorithm, and an algorithm is not a particular execution. Second, an actual execution contains more structure than a sequence of global states: it contains represented values, event occurrences, and provenance. Third, a modal mechanism is stronger than an actual occurrence because it specifies behavior under unrealized inputs or interventions. Fourth, computational descriptions form a partially ordered family of abstractions rather than a single scalar “level.”

The theory therefore has two realization layers. *Occurrent realization* maps a physical episode to a typed valued provenance structure using a coherent, trajectory-independent representation scheme. *Modal realization* maps a physical mechanism to an abstract mechanism while preserving admissible interventions. Their abstractions generate structured spectra. A final section formalizes one deliberately actualist candidate schema for computational functionalism. Its axioms are motivated defaults rather than consequences of the computational theory, and the consciousness-making property remains to be supplied by a substantive theory.

1 Preliminaries and design commitments

All structures are finite unless otherwise stated. This keeps the basic results elementary; measurable, stochastic, continuous-time, and infinite generalizations can be added later. We distinguish physical objects from abstract computational objects only by notation and typing, not by assuming a particular metaphysics of physical causation.

A *token* is an occurrence of a represented value. An *event* consumes zero or more tokens and produces zero or more tokens. The event-token incidence structure records actual provenance. It does not, by itself, assert counterfactual necessity or philosophical actual causation.

Intuition

The primitive occurrent question is: “what values were actually produced, transmitted, and used by what events during this episode?” The primitive modal question is stronger: “what would this organized mechanism do under other admissible inputs or interventions?”

2 Typed valued provenance structures

2.1 Bare provenance

Definition 2.1 (Typed port-labeled provenance structure). A *typed port-labeled provenance structure* is a tuple

$$P = (E, T, \Theta, \tau, \Pi^{\text{in}}, \Pi^{\text{out}}, I, O, \prec)$$

where:

- E is a finite set of event occurrences;
- T is a finite set of value-token occurrences;

- Θ is a set of token types and $\tau : T \rightarrow \Theta$ assigns a type to each token;
- Π_e^{in} and Π_e^{out} are finite typed input and output port sets for every $e \in E$;
- $I \subseteq T \times E \times \bigcup_e \Pi_e^{\text{in}}$ is the input-incidence relation, with $(t, e, p) \in I$ only when $p \in \Pi_e^{\text{in}}$;
- $O \subseteq E \times \bigcup_e \Pi_e^{\text{out}} \times T$ is the output-incidence relation, with $(e, p, t) \in O$ only when $p \in \Pi_e^{\text{out}}$;
- each occupied port is incident to at most one token, and each non-source token has at most one producing event-port pair;
- $\prec$ is a strict partial order on $E \cup T$ containing the causal order generated by $(t, e, p) \in I$, $(e, p, t) \in O$, and any additional physically justified precedence relations.

We require acyclicity on the finite episode.

The bipartite graph induced by I and O is the *provenance graph*. A token may fan out to several event ports; an event may have several inputs and outputs. Port labels make local tuples and their permutations explicit. The order $\prec$ allows additional causal/temporal precedence without forcing all concurrent events into one total order.

Persistent hardware identity may be retained as an annotation of a physical realization, but it is not a primitive of computational occurrence. Reusing one device for several event occurrences or replacing those uses by distinct devices does not by itself change the computation when the represented events, ports, values, and dependencies are preserved. Hardware persistence matters only when it induces some further computationally represented distinction admitted by the selected abstraction policy.

Definition 2.2 (Valued provenance structure). For each type $\theta \in \Theta$, let A_θ be an abstract value set. A *valued provenance structure* is a typed port-labeled provenance structure together with

$$\nu : T \rightarrow \bigsqcup_{\theta \in \Theta} A_\theta, \quad \nu(t) \in A_{\tau(t)}.$$

2.2 Physical carriers and coherent representation

Let a physical episode provide, for each token occurrence t , a local physical carrier state $c(t)$ in a carrier space $C_{\tau(t)}$. The carrier state may be a voltage pattern, spike configuration, magnetic state, optical mode, or any other physically specified local vehicle.

Definition 2.3 (Representation scheme). A *representation scheme* for token types Θ is a family of partial decoders

$$D = (D_\theta)_{\theta \in \Theta}, \quad D_\theta : C_\theta \rightharpoonup A_\theta.$$

It represents a physical token t by $\nu(t) = D_{\tau(t)}(c(t))$ whenever the decoder is defined.

Definition 2.4 (Coherence). A representation scheme is *coherent on an episode* if:

1. the same decoder D_θ is used for every token of type θ ;
2. D_θ depends only on the admissible local carrier state and fixed type/context data, not on the absolute time index, future history, target answer, or global trajectory identifier;
3. where a type is transported through identity-preserving channels, decoding is stable under the declared physical equivalence of carrier realizations;
4. the family D is fixed independently of the abstract trajectory one is attempting to prove after the fact.

The precise admissibility class of local carrier decompositions is domain-dependent. We therefore write $D \in \text{Adm}_D(P)$ and make the admissibility policy an explicit parameter of the theory.

Proposition 2.5 (Arbitrary time-indexed relabeling). *Let $(x_0, \dots, x_n)$ be any finite physical state sequence with pairwise distinct states and let $(q_0, \dots, q_n)$ be any abstract state sequence of the same length. If decoders may depend on the time index, then there exists a decoder $D(i, x_i) = q_i$ mapping the first sequence to the second.*

Proof. Define D exactly on the observed pairs (i, x_i) by $D(i, x_i) = q_i$. No structural constraint is imposed, so the construction is immediate. $\square$

Proposition 2.6 (Stationary local encoding obstruction). *Suppose a physical carrier state c of a fixed type θ occurs at two token occurrences t_1, t_2 with $c(t_1) = c(t_2) = c$. Under a coherent deterministic decoder D_θ , the two abstract values must coincide. Hence no target labeling with $\nu(t_1) \neq \nu(t_2)$ can be realized by that scheme.*

Proof. Both values equal $D_\theta(c)$. $\square$

Proposition 2.7 (Unique-carrier underdetermination). *Let $t_1, \dots, t_n$ be tokens of one type whose local carrier states $c(t_1), \dots, c(t_n)$ are pairwise distinct. For any target values $a_1, \dots, a_n$, there is a single stationary partial decoder satisfying $D(c(t_i)) = a_i$ for every i .*

Proof. Define D on the observed carrier states by the displayed assignments. Pairwise distinctness makes the definition functional, and the resulting decoder is not time-indexed. $\square$

Remark 2.8. The two propositions delimit what stationarity achieves. It blocks incompatible reuse of the same carrier state, but it does not constrain an episode whose exact local microstates never repeat. Coherence therefore defeats only time-indexed relabeling. A nontrivial realization claim additionally needs an independently specified admissibility class of carriers and decoders, using criteria such as physically natural coarse variables, bounded descriptive complexity, stability, locality, compositional reuse, provenance preservation, or—at the modal level—intervention preservation. Those criteria may themselves carry modal or normative commitments and must be stated rather than hidden in the word “representation.”

3 Occurrent computation

3.1 Abstract computational occurrences

Definition 3.1 (Computational occurrence). A *computational occurrence* is a valued provenance structure

$$C = (E_C, T_C, \Theta_C, \tau_C, \Pi_C^{\text{in}}, \Pi_C^{\text{out}}, I_C, O_C, \prec_C, \nu_C)$$

together with its port families and optional event-schema labels $\lambda : E_C \rightarrow \Lambda$ that identify only the event information licensed at the occurrent level. At minimum, an event label may record the actual port-ordered input-output tuple. Events may share a schema label without being performed by the same persistent piece of hardware. A total operation type is not required.

Thus an event may be recorded only as the actual fact

$$(a_1, \dots, a_k) \mapsto (b_1, \dots, b_m)$$

without asserting what would happen on any other input tuple.

3.2 Occurrence morphisms

Definition 3.2 (Provenance homomorphism). Let P and C be typed valued provenance structures. A *provenance homomorphism* $h : P \rightarrow C$ consists of maps on events, tokens, and event-local ports such that:

1. incidence is preserved through the corresponding port maps;
2. order is preserved: $x \prec_P y \Rightarrow h(x) \preceq_C h(y)$ whenever the images are distinct;
3. token types and represented values are preserved through the declared type/value maps;

4. any event-schema labels declared structurally relevant are preserved through their declared maps.

The morphism may identify multiple low-level events or tokens into one abstract event or token; this is the basic abstraction operation.

Definition 3.3 (Witness-full provenance quotient). A provenance homomorphism $h : P \rightarrow C$ is a *witness-full provenance quotient* when:

1. it is surjective on abstract events and tokens;
2. every input and output incidence of C is the image of an incidence of P ;
3. every strict precedence declared in C is witnessed by a precedence relation, or by a directed provenance path, between suitable members of the corresponding fibers in P ;
4. identifications inside a fiber satisfy the selected admissibility policy, which may additionally require connected fibers, boundary preservation, or event-schema compatibility.

Thus an abstraction may forget or identify concrete structure, but it may not manufacture an abstract operand, result, or dependency that has no concrete witness.

Definition 3.4 (Occurrent realization witness). Let P be an ambient physical episode equipped with carrier states. An *occurrent realization witness* of an abstract occurrence C in an ambient physical episode P is a tuple $(P' \hookrightarrow P, D, w)$ where P' is an admissible selected subepisode, D is a coherent representation scheme on P' , and

$$w : P'_D \twoheadrightarrow C$$

is an admissible witness-full provenance quotient. The selection P' may discard physical activity irrelevant to this witness, but it must be boundary-full for every retained event: all incident tokens and ports that the policy treats as operands or results must either be retained or explicitly exposed as boundary tokens. We write

$$P \models_{\text{occ}, w} C.$$

Surjectivity says every abstract event and token is actually witnessed. The low-level episode may contain extra physical detail that is quotiented away.

Definition 3.5 (Occurrent realization category). For a physical episode P , let $\text{Real}_{\text{occ}}(P)$ be the category whose objects are realization witnesses $W = (P' \hookrightarrow P, D, C, w)$ with $P \models_{\text{occ}, w} C$. An arrow $W \rightarrow W'$ is a pair (s, q) consisting of an admissible map $s : P'_D \rightarrow P''_D$ between the selected decoded subepisodes and an admissible abstraction $q : C \rightarrow C'$ such that the square

$$w' \circ s = q \circ w$$

commutes. Identities and composition are componentwise. The important special case of an abstraction of a fixed witness has $P' = P''$, $D = D'$, and $s = \text{id}$.

Intuition

The category, rather than only the set of abstract computations, preserves *witness information*. If two disjoint subsystems of P both realize isomorphic copies of the same computation, they are two objects/witnesses before quotienting by isomorphism. This matters for multiplicity and later subject-individuation questions.

Definition 3.6 (Occurrent spectrum). First forget witness multiplicity and preorder the abstract occurrences realized by P by existence of an admissible abstraction. Let $C \sim D$ when both $C \preceq D$ and $D \preceq C$. The *occurrent computational spectrum* $\text{Spec}_{\text{occ}}(P)$ is the resulting poset of mutual-abstraction classes. Abstractly isomorphic occurrences are necessarily in the same class, but the quotient is by mutual reachability rather than by isomorphism alone. We write

$$[C] \preceq [D]$$

when D refines C , equivalently when there is an admissible abstraction $D \rightarrow C$.

The spectrum is therefore a coarse summary of the richer witness category.

Theorem 3.7 (Downward closure). *If $P \models_{\text{occ}} C$ and there is an admissible abstraction $q : C \rightarrow C'$, then $P \models_{\text{occ}} C'$.*

Proof. If $w : P'_D \rightarrow C$ witnesses realization, then $q \circ w : P'_D \rightarrow C'$ is an admissible realization witness by closure of admissible morphisms under composition. $\square$

Corollary 3.8. *$\text{Spec}_{\text{occ}}(P)$ is downward closed under admissible abstraction.*

3.3 Why a state sequence is too weak

Let $\text{seq}(C)$ forget incidence and retain only a chosen total order of global states or values. This forgetful map is highly non-injective.

Proposition 3.9 (Provenance non-identifiability from state sequence). *There exist non-isomorphic computational occurrences C_1, C_2 with identical state/output sequences $\text{seq}(C_1) = \text{seq}(C_2)$ but different provenance graphs.*

Proof. Let C_1 generate each next token from the previous token through a chain of events. Let C_2 generate every displayed token independently from a stored script or common clock. Choose identical displayed values. The sequences agree, but one provenance graph is a chain and the other is a star or set of independent reads; they are not isomorphic. $\square$

This formalizes the movie-playback distinction: replaying snapshots is not automatically the same occurrent computation.

4 Local operations, partial laws, and value support

4.1 Partial operation tables from actual repetitions

Let C be a computational occurrence equipped with a partial event-schema labeling $\lambda : E_C \rightarrow \Lambda$. For a schema $\ell \in \Lambda$, collect the actual port-ordered input-output tuples observed across events with $\lambda(e) = \ell$ and a common typed port signature. The port structure is what makes repeated tuples comparable rather than equal only up to an unspecified permutation. Grouping events under ℓ is part of the representation/admissibility claim; it is not inferred merely from reuse of one physical device.

Definition 4.1 (Empirical partial operation). The *empirical partial operation* of schema ℓ in occurrence C is the partial relation/function

$$\widehat{f}_\ell : \prod_i A_i \rightarrow \prod_j B_j$$

whose graph consists exactly of the distinct actual input-output tuples observed at events labeled ℓ , provided repeated identical inputs have compatible outputs.

This object is strictly occurrent: it says nothing about input tuples not encountered.

4.2 Value supports

Now suppose a local event type is enriched with a total operation

$$f : \prod_{i=1}^n A_i \rightarrow B$$

and the actual input is $a = (a_1, \dots, a_n)$ with output $y = f(a)$.

Definition 4.2 (Value support). A subset $S \subseteq \{1, \dots, n\}$ is a *value support* for (f, a, y) if

$$\forall x \in \prod_i A_i : (\forall i \in S, x_i = a_i) \Rightarrow f(x) = y.$$

It is *minimal* if no proper subset has the property. The family of minimal supports is

$$\text{MinSupp}(f, a) = \{S : S \text{ is minimal support for } f(a)\}.$$

The minimal supports form a hypergraph over the actual input positions.

Example 4.3 (OR and AND). For $f = \text{OR}$ and $a = (1, 1, 0)$ with output 1,

$$\text{MinSupp}(f, a) = \{\{1\}, \{2\}\}.$$

For $g = \text{AND}$ and $a = (1, 1)$ with output 1,

$$\text{MinSupp}(g, a) = \{\{1, 2\}\}.$$

Proposition 4.4 (Support is not purely occurrent). *There exist total operations $f \neq g$ and an actual tuple a such that $f(a) = g(a)$ and the primitive occurrent provenance/value record at a is identical, while $\text{MinSupp}(f, a) \neq \text{MinSupp}(g, a)$.*

Proof. Take $f = \text{OR}$ and $g = \text{AND}$ at $a = (1, 1)$. Both output 1 on the actual tuple; the minimal supports differ as above. $\square$

Remark 4.5. Value support is *occurrence-indexed but locally modal*: it refers only to actual assignments in a , yet sufficiency quantifies over unrealized assignments of the other coordinates. It must therefore not be conflated with primitive provenance.

Proposition 4.6 (Boolean supports are certificates). *For a total Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and input a , a set S is a value support for $(f, a, f(a))$ exactly when the partial assignment $(x_i = a_i)_{i \in S}$ is a certificate for the value of f at a . Minimal value supports are therefore the inclusion-minimal certificates compatible with a .*

Proof. Both conditions say that every total input extending the same partial assignment has output $f(a)$. $\square$

4.3 Participation, support, contribution, dependence

We reserve four different relations:

1. **Participation/provenance:** token t was actually supplied to event e .
2. **Value support:** a set of actual assignments suffices for the actual output relative to a known local operation.
3. **Actual contribution:** an optional theory-specific relation $\text{Contrib}(t, e, y)$ intended to capture causal/explanatory contribution. The core theory leaves its axioms open.
4. **Modal dependence:** the total local mechanism function depends on a variable across possible inputs/interventions.

No theorem below requires identifying these relations.

5 Modal computational mechanisms

5.1 Open synchronous mechanisms

Definition 5.1 (Modular open mechanism). A deterministic synchronous *modular open mechanism* is a tuple

$$M = (V, U, Y, \{X_v\}_{v \in V}, G, \{f_v\}_{v \in V}, r)$$

where:

- V is a finite set of internal variables/components;
- U and Y are external input and output spaces;
- X_v is the state/value space of component v ;
- G specifies, for each v , a parent set $\text{Pa}(v) \subseteq V$ and selected input coordinates;
- each local update f_v maps current parent values and current external input to the next value of v ;
- r maps the current/next internal state and input to an external output.

Together the local functions induce a global synchronous transition/output function

$$F_M : X \times U \rightarrow X \times Y, \quad X = \prod_{v \in V} X_v.$$

This model captures open-system behavior while retaining internal factorization.

5.2 Interventions

An intervention may clamp variables, replace local mechanisms, set inputs, or perform another operation declared admissible for the system. Let $\mathcal{J}_M$ be a specified intervention language. Each $j \in \mathcal{J}_M$ induces an intervened mechanism M^j and global evolution F_M^j .

Definition 5.2 (Modal abstraction/realization). Let L be a low-level mechanism and H a high-level mechanism. A *modal realization morphism* $\rho : L \Rightarrow H$ consists of:

1. a grouping/assignment of low-level variables to high-level variables;
2. relevant state domains $Z_L \subseteq X_L$ and $Z_H \subseteq X_H$, and a decoder $\phi : Z_L \rightarrow Z_H$ compatible with that grouping;
3. an input decoder $\eta : U_L \rightarrow U_H$ and an output decoder $\chi : Y_L \rightarrow Y_H$;
4. an intervention translation $\omega : \mathcal{J}_H \rightarrow \mathcal{J}_L$;

such that for every admissible high-level intervention j , every low-level input u in the domain of η , and every $x \in Z_L$, the intervened low-level successor remains in Z_L and

$$\phi(\pi_X F_{L^{\omega(j)}}(x, u)) = \pi_X F_{H^j}(\phi(x), \eta(u)),$$

together with

$$\chi(\pi_Y F_{L^{\omega(j)}}(x, u)) = \pi_Y F_{H^j}(\phi(x), \eta(u)).$$

The translation preserves the identity intervention and every declared composition of compatible interventions. The representation/grouping must satisfy the same admissibility principles of locality, coherence, and non-ad-hoc construction used at the occurrent level.

This is a deterministic special case of intervention-preserving causal abstraction.

Definition 5.3 (Unfolding-compatible modal realization). A modal realization $\rho : L \Rightarrow H$ is *unfolding-compatible* when its variable grouping is equipped with local event and port witnesses such that, on every relevant run, each high-level update occurrence is realized by a nonempty connected low-level subcomputation, its boundary input/output tokens decode to the corresponding high-level ports, and these local witnesses jointly form a witness-full provenance quotient between the unrolled occurrences.

Definition 5.4 (Modal realization category and spectrum). $\text{Real}_{\text{mod}}(L)$ has objects $(L' \hookrightarrow L, H, \rho)$, where L' is an admissible selected submechanism of the ambient mechanism L and $\rho : L' \Rightarrow H$ is a modal realization morphism. An arrow between two such objects is a pair consisting of an admissible map between their selected low-level submechanisms and an admissible abstraction between their high-level mechanisms, with the resulting witness square commuting. Forgetting witness multiplicity gives a preorder of realized high-level mechanisms; quotienting by mutual abstraction gives the *modal computational spectrum* $\text{Spec}_{\text{mod}}(L)$.

Theorem 5.5 (Composition of modal realization). *If $L \Rightarrow H$ and $H \Rightarrow K$ are admissible modal realization morphisms, then their composite is an admissible modal realization $L \Rightarrow K$.*

Proof. Compose state, input, and output decoders, variable groupings, and the contravariant intervention translations. Restrict to the states on which both partial composites are defined. The required commuting equations follow by substitution of the two commuting equations. Admissibility and the relevant-domain conditions are assumed closed under composition. $\square$

Corollary 5.6. $\text{Spec}_{\text{mod}}(L)$ is downward closed under modal abstraction.

6 Relation between modal and occurrent realization

A concrete input stream and initial state determine a run of a modal mechanism. Unrolling the run yields an actual provenance structure if the internal event decomposition is retained.

Theorem 6.1 (Modal-to-occurrent projection). *Let $L \Rightarrow H$ be an unfolding-compatible modal realization morphism and let r be an actual run of L within its relevant domain under an admissible input/intervention sequence. Then the induced low-level occurrence realizes the corresponding unrolled high-level occurrence.*

Proof sketch. Restrict the modal commuting diagrams and the stipulated local witnesses to the finite sequence traversed by r . State, input, and output commutation gives the token values. Unfolding compatibility supplies the event/port mapping and guarantees incidence fullness and boundary preservation. Their union is therefore a witness-full provenance quotient onto the unrolled high-level occurrence. $\square$

Remark 6.2. State-level commutation alone is insufficient for this theorem: two mechanisms can induce the same global transition while having non-isomorphic internal provenance. Unfolding compatibility is the additional local hypothesis that connects modal state abstraction to occurrence-level event structure.

Theorem 6.3 (Occurrent underdetermination of modal mechanism). *There exist modal mechanisms $M_1 \not\equiv M_2$ and runs r_1, r_2 whose induced computational occurrences are isomorphic.*

Proof. Take two one-step Boolean gates OR and AND and the actual input $(1, 1)$. Both produce the same actual tuple $(1, 1) \mapsto 1$ with identical primitive provenance, while their modal transition tables differ on $(1, 0)$ and $(0, 1)$. $\square$

Corollary 6.4. *The projection from compatible modal mechanisms to occurrent episodes is generally non-injective: a finite actual episode determines a set of compatible modal completions rather than a unique modal mechanism. This is an information-loss claim, not the claim that some occurrence lacks every modal completion.*

Definition 6.5 (Compatible modal completion). For an occurrent structure C , a modal mechanism M is a *compatible completion* if some run of M induces an occurrence isomorphic to C . Let

$$\text{Comp}(C) = \{[M] : M \text{ is a compatible modal completion of } C\}.$$

Repeated actual instances of the same admissibly grounded event schema shrink $\text{Comp}(C)$ by expanding its empirical partial operation, but cannot in general make the completion unique without enough coverage or extra assumptions.

7 Abstraction geometry: refinement is not a scalar

7.1 Refinement order

Before poset reflection, define $A \preceq B$ when there exists an admissible abstraction $B \rightarrow A$. Thus B is at least as fine as A .

Proposition 7.1 (Poset reflection). *If admissible abstractions contain identities and are closed under composition, $\preceq$ is a preorder. Define $A \sim B$ iff $A \preceq B$ and $B \preceq A$. The induced relation on mutual-abstraction classes is a partial order.*

Proof. Identity gives reflexivity and composition gives transitivity. Mutual reachability is an equivalence relation, and quotienting by it makes the induced preorder antisymmetric. $\square$

7.2 Independent local abstractions

Let a computation be a composition of components A and B . Suppose component A admits abstraction poset L_A and B admits L_B , the choices can be made independently while preserving the external composition interface, and distinct pairs of local abstraction classes remain distinct as global abstraction classes.

Theorem 7.2 (Product abstraction structure). *Under the independence and faithfulness assumptions, the combined abstraction poset contains a subposet isomorphic to $L_A \times L_B$ with componentwise order.*

Proof. Map a pair of local abstractions (a, b) to the global computation obtained by applying a to A and b to B . Independence gives well-defined composition. Refinement in one component while fixing the other induces the corresponding global refinement. Faithfulness makes the map injective on classes, and the component interfaces recover the two local choices on its image. $\square$

Corollary 7.3 (Non-linearity of granularity). *If both L_A and L_B contain a strict two-element chain, the global abstraction order contains incomparable elements: “ A abstract, B detailed” and “ A detailed, B abstract.” Hence computational granularity cannot in general be represented by one scalar level.*

7.3 Lattice conditions

Proposition 7.4. *If each local abstraction order L_i is a lattice and local abstraction choices combine independently, then the product $\prod_i L_i$ is a lattice with meet and join computed componentwise.*

The global spectrum need not be a lattice when abstractions interact or admissibility fails to compose. “Lattice” is therefore a theorem under conditions, not part of the definition.

8 Function, algorithm, and nested substitution

Definition 8.1 (Boundary transduction). For an open mechanism M , its *boundary transduction* is the map from admissible initial state/input histories to output histories after hiding internal variables.

Two mechanisms can have equal boundary transductions while differing internally. Extensional equality is therefore a coarse abstraction.

Example 8.2 (Triangular numbers). An iterative algorithm computing $1 + \dots + n$ and a closed-form algorithm computing $n(n+1)/2$ have the same total function on n but non-isomorphic fine-grained occurrences and mechanisms.

Definition 8.3 (Contextual interface equivalence). Let A and B be components exposing the same interface Γ . They are Γ -*equivalent* if no admissible interaction expressible through Γ distinguishes them at the specified occurrent or modal level.

Theorem 8.4 (Substitution/congruence). *Let $K[-]$ be a context whose only interaction with its hole is through interface Γ . If $A \equiv_{\Gamma} B$ and the equivalence is a congruence for the composition operator, then*

$$K[A] \equiv K[B].$$

This theorem formalizes the case in which a larger program treats either summation routine as the same abstract subroutine even though their internals differ.

9 Cross-system spectral relations

Let $\mathfrak{C}$ be a universe of abstract computation isomorphism classes for a fixed admissibility policy.

Definition 9.1 (Spectral containment). For systems/episodes P, Q at the same layer $\ell \in \{\text{occ}, \text{mod}\}$,

$$P \sqsubseteq_{\ell} Q \iff \text{Spec}_{\ell}(P) \subseteq \text{Spec}_{\ell}(Q).$$

Definition 9.2 (Spectral equivalence). $P \equiv_{\ell}^{\text{spec}} Q$ iff $P \sqsubseteq_{\ell} Q$ and $Q \sqsubseteq_{\ell} P$.

Definition 9.3 (Common computational core). The *common core* is

$$\text{Core}_{\ell}(P, Q) = \text{Spec}_{\ell}(P) \cap \text{Spec}_{\ell}(Q).$$

For a family $\mathcal{F}$, define $\text{Core}_{\ell}(\mathcal{F}) = \bigcap_{P \in \mathcal{F}} \text{Spec}_{\ell}(P)$.

These set-level relations intentionally forget witness multiplicity. When multiplicity matters, comparison should be performed in $\text{Real}_{\ell}(P)$ and $\text{Real}_{\ell}(Q)$ using embeddings, spans, or correspondences between witness categories.

Proposition 9.4 (Containment is a preorder on systems). $\sqsubseteq_{\ell}$ is reflexive and transitive. Quotienting systems by spectral equivalence yields a partial order.

10 A stronger anti-triviality package

No single anti-triviality axiom is sufficient in all physical domains. The theory therefore treats admissibility as a conjunction of constraints rather than a single decisive mapping restriction.

Definition 10.1 (Admissible occurrent realization policy). An admissibility policy is an explicit, target-independent rule fixing the eligible physical factorization, carrier variables, decoder family, selected subepisodes, and quotient morphisms before a target trajectory is fitted. It must state how it enforces or replaces each of the following constraints:

1. **Stationarity/coherence:** decoders do not vary ad hoc by time or target trajectory.
2. **Locality:** represented values are decoded from declared carriers or bounded local structures, not from the entire history.
3. **Incidence and boundary fullness:** abstract dependencies are witnessed by actual production/use relations, and selecting a subepisode cannot silently delete a retained event's relevant operands or results.
4. **Compositionality:** local encodings compose across repeated types and subcomputations.
5. **No answer smuggling:** the decoder may not contain the target output/factorization as an oracle unless that information is physically present in the declared carrier structure.
6. **Physical regularity:** carriers and event decompositions satisfy domain-specific stability or locality constraints.

Definition 10.2 (Admissible modal realization policy). A modal policy adds intervention preservation across a declared intervention family, ruling out mappings that fit only the realized trajectory.

Scope condition

The framework does not claim that there is a unique metaphysically privileged admissibility class for all sciences. Its objectivity claim is conditional but non-arbitrary: once a policy is fixed independently of the desired computation, realization and non-realization are mathematical facts. Implementation assertions that suppress the policy are incomplete, and many classic triviality constructions exploit precisely the omission of locality, stationarity, compositionality, provenance, physical regularity, or intervention constraints.

11 Resource-sensitive extension

Let $R(M) \in \mathbb{R}_+^d$ be a resource vector for a realization: description length, memory, circuit size, depth, time, energy, bandwidth, locality cost, or any chosen coordinates. For a target abstract computation C and resource region $B \subseteq \mathbb{R}_+^d$, define

$$\mathcal{R}(C; B) = \{M : M \models C, R(M) \in B\}.$$

Definition 11.1 (Resource-constrained computational core).

$$\text{Core}(C; B) = \bigcap_{M \in \mathcal{R}(C; B)} \text{Spec}(M),$$

when the realization family is nonempty.

Theorem 11.2 (Budget monotonicity). *Assume that both realization families are nonempty. If $B_1 \subseteq B_2$, then*

$$\text{Core}(C; B_1) \supseteq \text{Core}(C; B_2).$$

Proof. $\mathcal{R}(C; B_1) \subseteq \mathcal{R}(C; B_2)$. Intersecting spectra over a smaller family can only enlarge the intersection. $\square$

This is the rigorous form of the intuition that tightening resources can make some internal computational structure unavoidable. It does *not* imply convergence to the structure of a chosen reference implementation.

If every admissible spectrum contains a trivial bottom abstraction, then that object also lies in every common core and carries no rigidity information. Applications should therefore report nontrivial maximal common elements, rank-filtered cores, or a baseline-subtracted core rather than treating nonempty intersection by itself as significant.

Definition 11.3 (Reference-relative overlap profile). For a reference realization M_0 of C , let $B(b)$ be the resource region induced by a scalar threshold b . For a chosen overlap measure μ , define

$$\mathcal{O}_{M_0, C}(b) = \inf_{M \in \mathcal{R}(C; B(b))} \mu(\text{Spec}(M_0) \cap \text{Spec}(M)).$$

The shape of this profile expresses spectral rigidity or degeneracy under compression/resource constraints.

12 A candidate schema for actual-process computational functionalism

This section does not derive psychophysical constraints from the computational theory. It defines one deliberately actualist subclass of computational-functionalism theories by turning the companion note's naturalness desiderata into explicit axioms. These axioms are motivated defaults: a different computational functionalism may reject one, but then lies outside this candidate subclass and should state the resulting modal, representational, or individuation commitment explicitly.

12.1 Phenomenal assignment

Let $\mathcal{E}$ be a class of physical episodes and $\mathcal{H}$ a set of phenomenal states, phenomenal structures, or equivalence classes thereof. A computational-functionalism theory supplies a partial map

$$\Phi : \mathcal{E} \rightarrow \mathcal{H}$$

whose domain contains candidate conscious episodes. The substantive consciousness theory specifies which computational invariant determines Φ . Within the candidate subclass defined here, the following axioms stipulate invariances of that dependence.

Let $\mathcal{A}(P)$ denote the consciousness-relevant actual computational structure extracted from $\text{Real}_{\text{occ}}(P)$ or $\text{Spec}_{\text{occ}}(P)$. The extraction rule may retain some abstractions and discard others. To belong to the present candidate class, it must satisfy the invariance axioms below.

Three roles of modal information must be distinguished. Modal facts may help *individuate* the carriers, variables, operations, or boundaries entering $\mathcal{A}(P)$; interventions may provide *evidence* for that individuation; and a theory may or may not treat unmanifested alternatives as *constitutive* of present phenomenal character. F2 constrains only the third role. It does not license an arbitrary occurrent decoding, and it cannot be invoked until the grounding of $\mathcal{A}(P)$ has been fixed independently.

Let $\mathcal{N}$ be a theory-specified family of neutral redescriptions and refinements. Membership in $\mathcal{N}$ must be stated using computational or physical criteria rather than by appealing to the value of Φ that the invariance is meant to explain.

F1. Phenomenal Actuality

If two episodes have isomorphic consciousness-relevant actual computational structures,

$$\mathcal{A}(P) \cong \mathcal{A}(Q),$$

then $\Phi(P) = \Phi(Q)$, absent an independently stated non-computational variable.

F2. Counterfactual Irrelevance

Let mechanisms M_1, M_2 generate episodes P_1, P_2 with independently grounded structures $\mathcal{A}(P_1) \cong \mathcal{A}(P_2)$. If M_1 and M_2 differ only on modal transitions not instantiated in those episodes, and that difference plays no role in individuating any carrier, event, representation, boundary, or actual dependency retained in $\mathcal{A}$, then the unmanifested difference alone does not alter Φ :

$$\Phi(P_1) = \Phi(P_2).$$

F3. Representation Invariance

If Q is obtained from P by a coherent bijective recoding of represented values and corresponding relabeling of event types that preserves the actual computational structure, then $\Phi(Q) = \Phi(P)$.

F4. Substrate Invariance

If two physical episodes realize the same consciousness-relevant actual computational structure under admissible realization witnesses, differences in substrate alone do not change Φ .

F5. Inert-extension Invariance

If Q is obtained from P by adjoining physical or modal structure that is causally isolated from the declared bearer boundary throughout the relevant interval—not merely omitted from a chosen witness—then $\Phi(Q) = \Phi(P)$.

F6. Abstraction Consistency

Replacing one spectral description of a fixed physical realization by an isomorphic description or by a refinement in $\mathcal{N}$ does not create a new phenomenal episode. Phenomenal multiplicity is indexed by realization witnesses/subsystems, not by the number of descriptions in the spectrum.

F7. Capacity–Occurrence Separation

Phenomenal capacity is permitted to depend on $\text{Spec}_{\text{mod}}(M)$, but present phenomenal character $\Phi(P)$ is determined by the actual episode through $\mathcal{A}(P)$ unless the theory explicitly promotes a modal property to constitutive status.

12.2 Generative sensitivity is permitted, not snapshot identity

A theory satisfying F1–F7 may distinguish two episodes with identical snapshot sequences if their actual provenance differs. Formally, Φ is not required to factor through the forgetful map $P \mapsto \text{seq}(P)$; it is instead allowed to factor through the richer occurrent realization structure.

Proposition 12.1 (Movie playback is not forced equivalent). F1–F7 do not imply phenomenal equivalence between an actually generated computational chain and a scripted replay with the same state sequence when their consciousness-relevant provenance structures are non-isomorphic.

Proof. F1 applies only when the consciousness-relevant actual computational structures are isomorphic. Identical state sequences do not imply provenance isomorphism by the provenance non-identifiability proposition above. $\square$

12.3 OR-to-AND off-trajectory replacement

Proposition 12.2 (Off-trajectory gate replacement). Suppose an episode P contains a gate occurrence with actual input $(1, 1)$ and output 1, and Q is identical in consciousness-relevant actual provenance/-

value structure except that the underlying total local mechanism is changed from OR to AND. If the total operation type and value-support hypergraph are not included in $\mathcal{A}$, then F2 entails $\Phi(P) = \Phi(Q)$.

Proof. The actual computational structure $\mathcal{A}$ is unchanged by hypothesis. OR and AND differ only on unrealized local inputs for this occurrence. F2 applies. $\square$

Remark 12.3. If a candidate theory includes value-support structure or local operation type in $\mathcal{A}$, then the two episodes may differ. The burden then shifts to that theory to justify why the locally counterfactual distinction is constitutive of present phenomenology rather than merely diagnostic of modal capacity.

12.4 Actual-process functionalism

Definition 12.4 (Actual-process computational functionalism). A computational-functionalist theory is *actual-process* if Φ factors through an invariant of $\text{Real}_{\text{occ}}(P)$ (possibly after admissible quotienting) and satisfies F2: purely off-trajectory modal changes do not alter present phenomenal character.

The substantive theory selects the consciousness-making invariant. This definition stipulates a particular actualist type of dependence; it is not a theorem about every possible computational functionalism.

13 Open problems exposed by the formalism

13.1 Admissible factorization and variable choice

The theory assumes an admissible decomposition into carriers, tokens, and event occurrences. Arbitrary coordinate transformations can destroy locality or manufacture it. A mature physical theory must therefore define admissible factorization using independently motivated physical criteria: locality, causal separability, stability, interaction boundaries, or dynamical modularity. Persistent physical components may help justify that decomposition or a shared operation schema, but their material identity is not by itself a computational invariant.

13.2 Subject individuation and overlapping witnesses

$\text{Spec}_{\text{occ}}(P)$ alone loses multiplicity. The witness category $\text{Real}_{\text{occ}}(P)$ retains it, but a consciousness theory still needs a rule deciding when two overlapping consciousness-bearing witnesses correspond to one subject, two subjects, or nested descriptions of one subject. Subject individuation is therefore a distinct layer of the theory.

The partial map $\Phi : \mathcal{E} \multimap \mathcal{H}$ is consequently only a one-bearer simplification. A fuller theory may require a bearer relation assigning zero, one, or several subject-indexed phenomenal episodes to one physical episode, together with consistency conditions under restriction and overlap.

13.3 Temporal metric

The provenance order records order and partial concurrency but not necessarily metric time. A theory may enrich events with durations or delays. Whether uniform slowdown, timing jitter, or temporal rescaling changes computational or phenomenal identity depends on which temporal quantities are retained by the relevant abstraction.

13.4 Approximate and stochastic realization

Exact commutation is too strong for noisy brains and approximate simulations. One extension replaces equality by a metric/divergence bound and exact morphisms by ε -realizations. Stochastic mechanisms replace local functions with kernels and require intervention-preserving equality or approximation of distributions.

Identical realized sample paths can have radically different probabilities under two kernels. The framework must therefore decide separately whether transition probabilities help individuate the actual computation and whether they are constitutive of an occurrent property; path identity alone does not settle either question.

13.5 Absence, inhibition, and alternative enabling

A false value carried by a low-voltage token is not the same as the absence of any token. Neural silence, timeouts, vetoes, prevention, and events enabled by either of several alternative histories require explicit interval, absence, or enabling structure. A bare positive incidence DAG cannot represent all of these cases without first reifying absences as independently justified tokens. General or stable event structures, occurrence nets, or an explicit enabling relation are candidate extensions.

13.6 Bearer boundaries and temporal composition

Selecting an episode fixes spatial, functional, and temporal boundaries. A mature theory must specify when upstream history, body, or environment belongs to the realization and how adjacent episodes compose. It should also state whether uniform slowdown, jitter, phase relations, or resampling preserve identity. These are not mere measurement details: changing the cut can change provenance, representation, and the number of candidate witnesses.

14 Compact axiomatics

The computational theory can be reconstructed from the following scheme.

1. A physical episode is decomposed into local carriers, token occurrences, event occurrences, and actual provenance.
2. Persistent identity of the realizing hardware is not a primitive computational invariant; it matters only through a further represented difference retained by the realization policy.
3. A coherent representation scheme assigns abstract values to carrier states without trajectory-indexed relabeling.
4. An occurrent computation is a typed valued provenance quotient witnessed by an admissible, witness-full morphism.
5. The category of all such witnesses induces a preorder of abstractions; quotienting by mutual abstraction gives the occurrent spectrum as a poset.
6. Repeated actual instances of an admissibly grounded event schema induce partial operation tables; total local operation semantics may optionally induce value-support hypergraphs.
7. A modal mechanism specifies total transition rules and admissible interventions. Modal realization requires abstraction to commute across those interventions.
8. Every unfolding-compatible modal realization projects to corresponding occurrent realizations on actual runs; the projection is generally non-injective.
9. Local abstractions compose when their interfaces support congruence; independent local choices generate product-like, non-linear refinement orders.

10. Cross-system relations are defined by spectral containment, equivalence, and common core, while witness categories preserve multiplicity.
11. Resource constraints define nested families of realizations and monotone resource-constrained common cores.
12. A deliberately actualist computational functionalism assigns present phenomenality to independently grounded invariants of actual computational realization and treats purely unrealized modal differences as non-constitutive by default, without denying their possible individuating or evidential roles.

One-line formal picture

For a physical episode P and mechanism M ,

$$\text{Real}_{\text{occ}}(P) \longrightarrow \text{Spec}_{\text{occ}}(P), \quad \text{Real}_{\text{mod}}(M) \longrightarrow \text{Spec}_{\text{mod}}(M),$$

with unfolding-compatible modal realization projecting to occurrent realization on each run, spectra ordered by admissible abstraction after poset reflection, and the actual-process functionalist candidate represented by invariant maps on the witness/spectral structure of actual episodes rather than by arbitrary state labeling or bare input-output equality.

Selected references

- [1] D. J. Chalmers. On implementing a computation. *Minds and Machines*, 4:391–402, 1994.
- [2] D. J. Chalmers. Does a rock implement every finite-state automaton? *Synthese*, 108:309–333, 1996.
- [3] G. Piccinini. *Physical Computation: A Mechanistic Account*. Oxford University Press, 2015.
- [4] N. G. Anderson and G. Piccinini. *The Physical Signature of Computation*. Oxford University Press, 2024.
- [5] Y. Gurevich. Sequential Abstract-State Machines capture sequential algorithms. *ACM TOCL*, 1(1):77–111, 2000.
- [6] A. Blass, N. Dershowitz, and Y. Gurevich. When are two algorithms the same? *Bulletin of Symbolic Logic*, 15(2):145–168, 2009.
- [7] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis. *POPL*, 1977.
- [8] G. Winskel. Event structures. LNCS 255, Springer, 1987.
- [9] H. Agrawal and J. R. Horgan. Dynamic program slicing. *PLDI*, 1990.
- [10] J. Cheney. Causality and the semantics of provenance. arXiv:1004.3241, 2010.
- [11] S. Beckers and J. Y. Halpern. Abstracting causal models. *AAAI*, 2019.
- [12] A. Geiger et al. Causal abstraction: a theoretical foundation for mechanistic interpretability. *JMLR*, 26(83):1–64, 2025.
- [13] S. Ma and R. Kanai. Intrinsic Computational Functionalism. arXiv:2606.06424, 2026.
- [14] R. Kanai and S. Ma. Intrinsic Computational Functionalism and simulated consciousness. arXiv:2606.15348, 2026.